

Retrieval Augmented generation (RAG)

Agenda of this Course:

- Introduction to RAG
- RAG Architecture
- End to End RAG Pipeline
- Advantage & Disadvantage of RAG
- Build RAG from Scratch
- Multimodel RAG
- RAG vs Fine Tuning.
- Evaluation Metrics of RAG
- How to Improve the RAG System and Important research paper.

1. Introduction to RAG (Retrieval Augmented Generation)

RAG is an AI framework for retrieving facts from an external knowledge base to ground large language models (LLMs) on the most accurate, up-to-date information and to give users insight into LLMs' generative process.

RAG is an architecture used to help LLM models like GPT-4, LLAMA, Gemini to provide a better response by using relevant information from additional sources and reduces the chance that LLM will lead to incorrect information.

RAG is a technique that enhances language model generation by incorporating external knowledge. This is typically done by Retrieving relevant information from a large corpus of documents and using that information to inform the generation training.

With RAG, the LLM is able to leverage knowledge and information that is not necessarily in its weights means it is not inside the training.

Why is Retrieval-Augmented Generation important?

LLMs, or Large Language Models, are powerful AI technologies that drive intelligent chatbots and other natural language processing applications. Their aim is to enable bots to respond to user queries across different contexts by referencing reliable knowledge sources.

However, LLMs come with certain drawbacks:

- They may provide incorrect information when it does not have the answer..
- They might offer outdated or general information when the user expects specific and up-to-date responses.
- Creating a response from non-authoritative sources.

- They may produce inaccurate responses due to confusion over terminology, where different sources use the same terms to refer to different things.

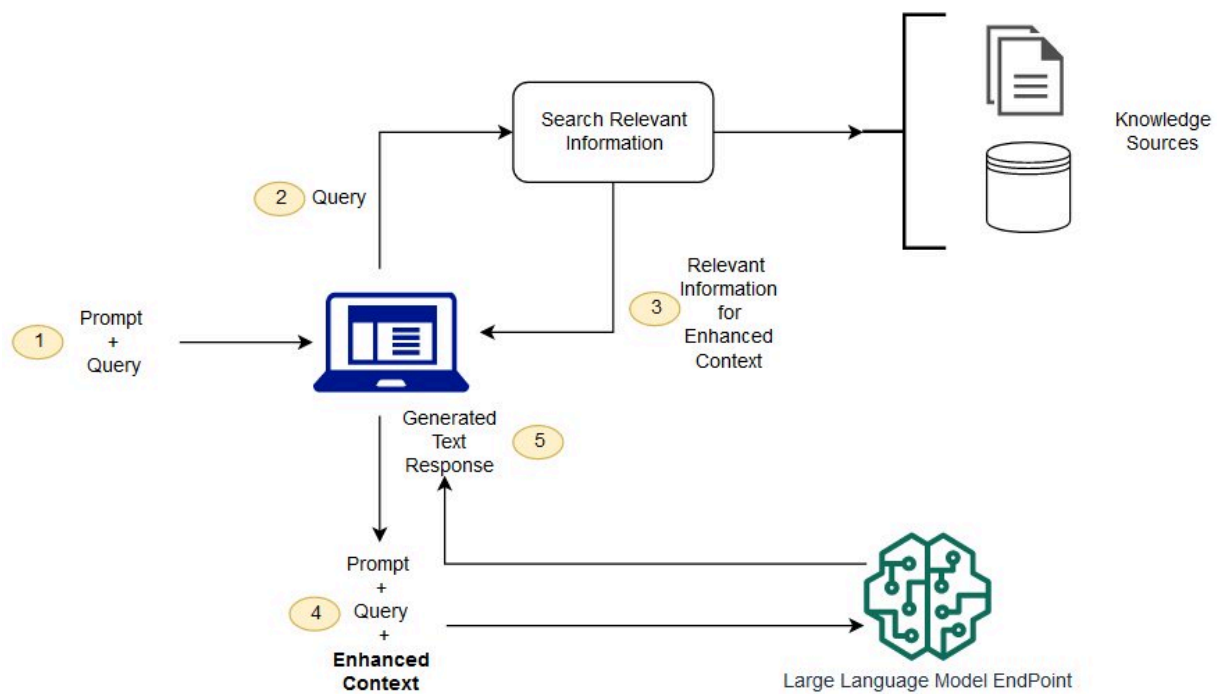
You can think of the Large Language Model as an over-enthusiastic new employee who refuses to stay informed with current events but will always answer every question with absolute confidence. Unfortunately, such an attitude can negatively impact user trust and is not something you want your chatbots to emulate!

To address these challenges, the **Retrieval-Augmented Generation** (RAG) approach is used. RAG directs LLMs to fetch relevant information from trusted, predetermined knowledge sources. This gives organizations more control over the generated responses and helps users understand how the LLM arrives at its answers.

What are the benefits of Retrieval-Augmented Generation?

1. Cost-effective implementation
2. Providing up-to-date Current information and accurate responses
3. Providing domain-specific, relevant responses & Enhanced user trust
4. Reducing inaccurate responses, or hallucinations
5. More developer control

2. RAG (Retrieval Augmented Generation) Architecture



1. Ingestion / Indexing
2. Retrieval
3. Generation

Indexing A pipeline that ingests data from a source and indexes it. This data can be structured or unstructured.

Retrieval and generation This is the actual RAG chain. It takes the user query and retrieves similar data from the index, then passes the data, along with the query, to the LLM model.

Step-wise-step Exploration

Indexing pipeline in a RAG app

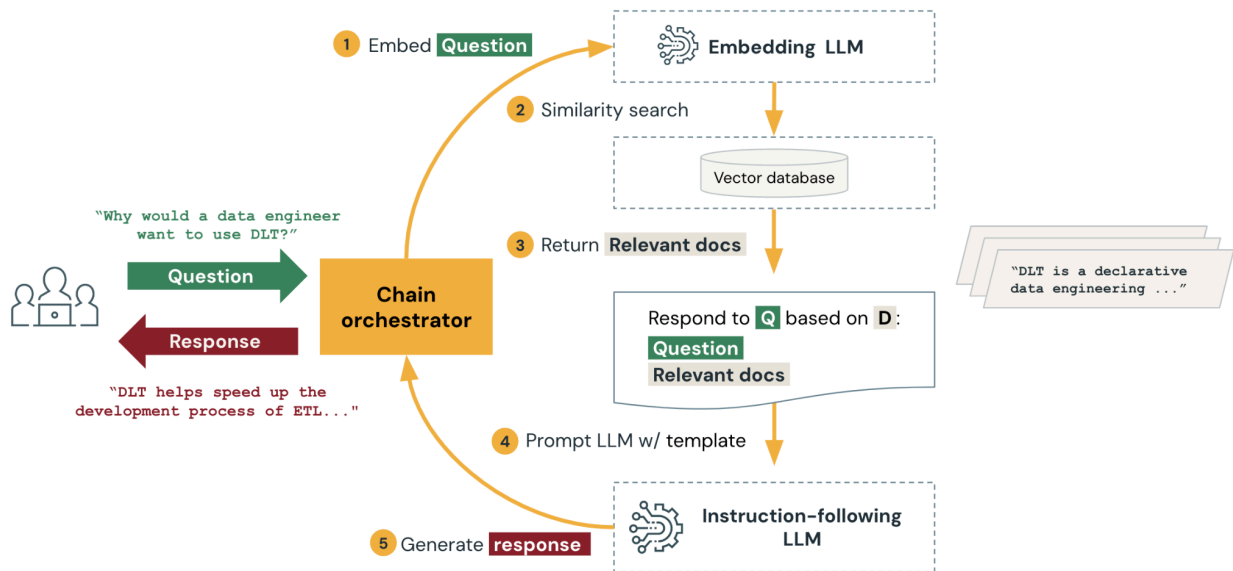
The following steps describe the indexing pipeline:

1. Ingest data from your proprietary data source.
2. Split the data into chunks that can fit into the context window of the foundational LLM. This step also includes parsing the data and extracting metadata. This data is commonly referred to as a knowledge base that the foundational LLM is trained on.
3. Use an embedding model to create vector embeddings for the data chunks.
4. Store the embeddings and metadata in a vector database to make them accessible for querying by the RAG chain.

Retrieval using the RAG chain

After the index is prepared, the RAG chain of the application can be served to respond to questions. The following steps and diagram describe how the RAG application responds to an incoming request.

1. Embed the request using the same embedding model that was used to embed the data in the knowledge base.
2. Query the vector database to do a similarity search between the embedded request and the embedded data chunks in the vector database.
3. Retrieve the data chunks that are most relevant to the request.
4. Feed the relevant data chunks and the request to a customized LLM. The data chunks provide context that helps the LLM generate an appropriate response. Often, the LLM has a template for how to format the response.
5. Generate a response.



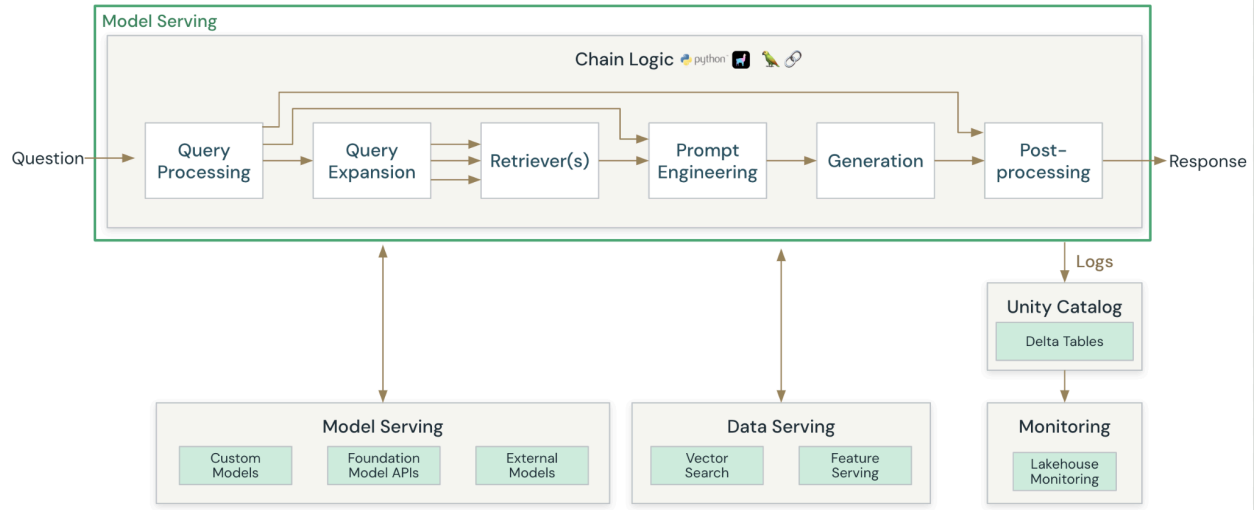
RAG chain

After the index is prepared, the RAG chain of the application can be served to respond to questions.

The following steps and diagram describe how the RAG chain operates in response to an incoming question.

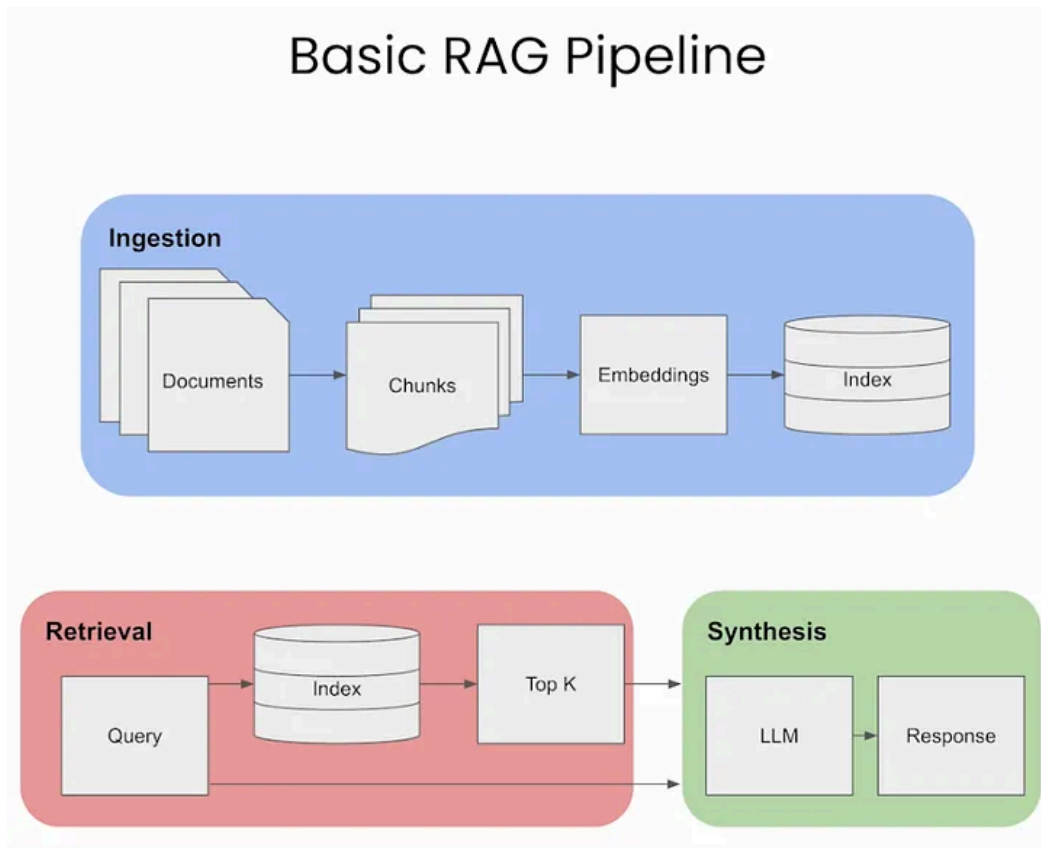
1. The incoming question gets embedded using the same embedding model that was used to embed the data in the knowledge base. Model Serving is used to serve the embedding model.
2. After the question is embedded, you can use Vector Search to do a similarity search between the embedded question and the embedded data chunks in the vector database.
3. After Vector Search retrieves the data chunks that are most relevant to the request, those data chunks along with relevant features from Feature Serving and the embedded question are consumed in a customized LLM for post processing before a response is generated.
4. The data chunks and features provide context that help the LLM generate an appropriate response. Often, the LLM has a template for how to format the response. Once again, Model Serving is used to serve the LLM. You can also use Unity Catalog and Lakehouse Monitoring to store logs and monitor the chain workflow, respectively.
5. Generate a response.

RAG Chain



Building a END to END RAG Pipeline

Basic RAG Pipeline



Ingestion

Chunking

- Chunking means breaking down prompts or documents into smaller parts called chunks. This makes them easier to handle. These chunks can be defined either by a fixed size, such as a specific number of characters, sentences or paragraphs.
- In RAG, each chunk is encoded into an embedding vector for retrieval. Smaller, more precise chunks lead to a finer match between the user's query and the content, enhancing the accuracy and relevance of the information retrieved.
- Larger chunks might include irrelevant information, introducing noise and potentially reducing the retrieval accuracy. By controlling the chunk size, RAG can maintain a balance between comprehensiveness and precision.
- So the next natural question that comes up is, how do you choose the right chunk size for your use case? The choice of chunk size in RAG is crucial. It needs to be small enough to ensure relevance and reduce noise but large enough to maintain the context's integrity. Let's look at a few methods below referred from [Pinecone](#):

Embeddings

- Once you have your prompt chunked appropriately, the next step is to embed it. Embedding prompts and documents in RAG involves transforming both the user's query (prompt) and the documents in the knowledge base into a format that can be effectively compared for relevance. This process is critical for RAG's ability to retrieve the most relevant information from its knowledge base in response to a user query.

Here's how it typically works:

One option to help pick which embedding model would be best suited for your task is to look at [HuggingFace's Massive Text Embedding Benchmark \(MTEB\) leaderboard](#).

There is a question of whether a dense or sparse embedding can be used so let's look into benefits of each below:

- **Sparse embedding:** Sparse embeddings such as TF-IDF are great for lexical matching the prompt with the documents. Best for applications where keyword relevance is crucial. It's computationally less intensive but may not capture the deeper semantic meanings in the text.
- **Semantic embedding:** Semantic embeddings, such as BERT or SentenceBERT lend themselves naturally to the RAG use case.

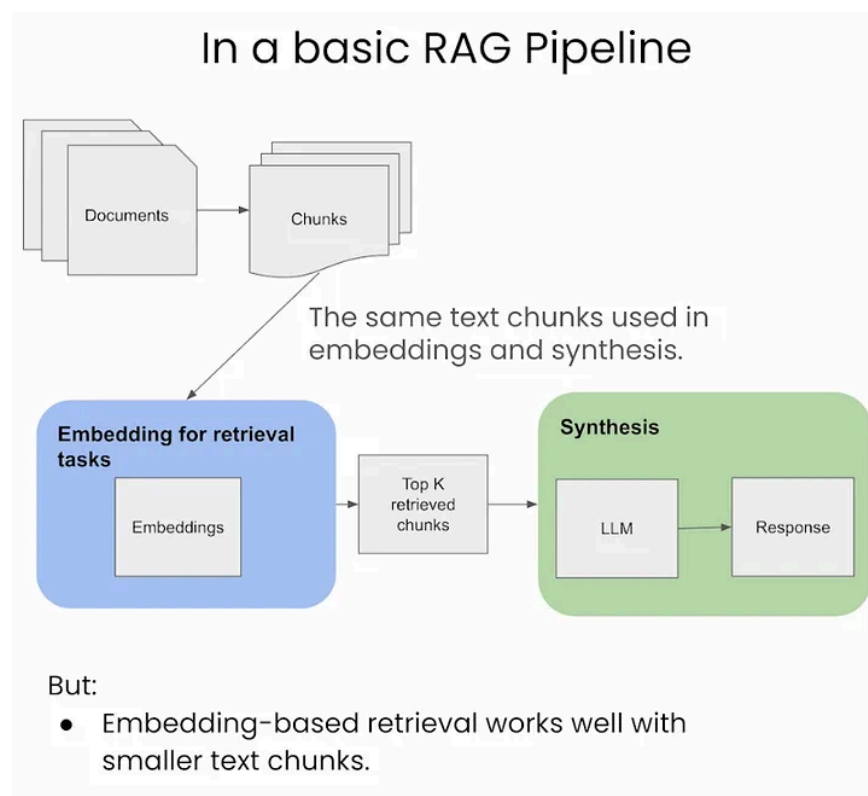
- **BERT:** Suitable for capturing contextual nuances in both the documents and queries. Requires more computational resources compared to sparse embeddings but offers more semantically rich embeddings.
- **SentenceBERT:** Ideal for scenarios where the context and meaning at the sentence level are important. It strikes a balance between the deep contextual understanding of BERT and the need for concise, meaningful sentence representations. This is usually the preferred route for RAG

Retrieval

Let's look at two different types of retrieval: **standard**, **sentence window**, and **auto-merging**. Each of these approaches has specific strengths and weaknesses, and their suitability depends on the requirements of the RAG task, including the nature of the dataset, the complexity of the queries, and the desired balance between specificity and contextual understanding in the responses.

1. Standard/Naive Approach

The standard pipeline uses the same text chunk for indexing/embedding as well as the output synthesis.



In the context of Retrieval-Augmented Generation (RAG) in Large Language Models (LLMs), here are the advantages and disadvantages of the three approaches:

Advantages

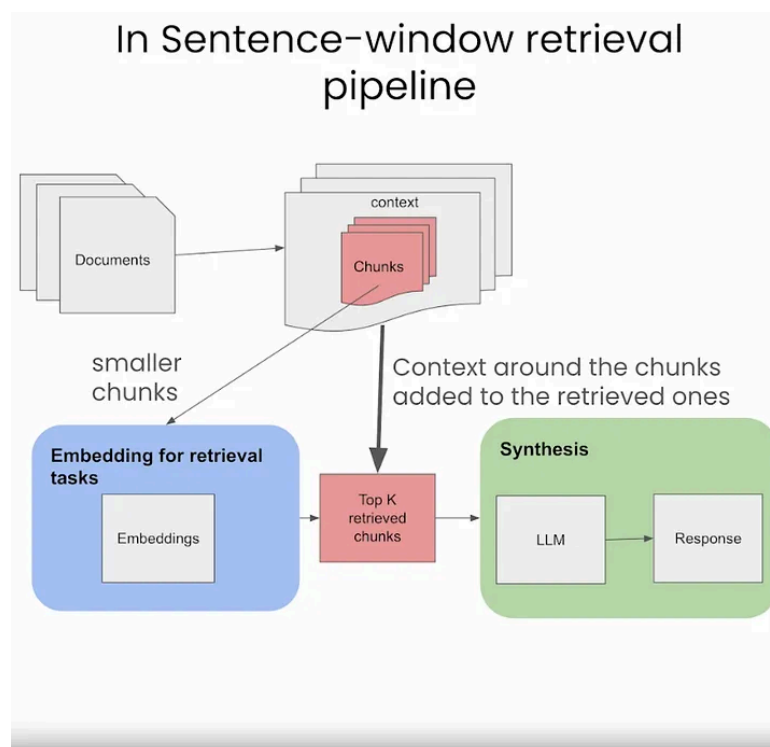
1. **Simplicity and Efficiency:** This method is straightforward and efficient, using the same text chunk for both embedding and synthesis, simplifying the retrieval process.
2. **Uniformity in Data Handling:** It maintains consistency in the data used across both retrieval and synthesis phases.

Disadvantages

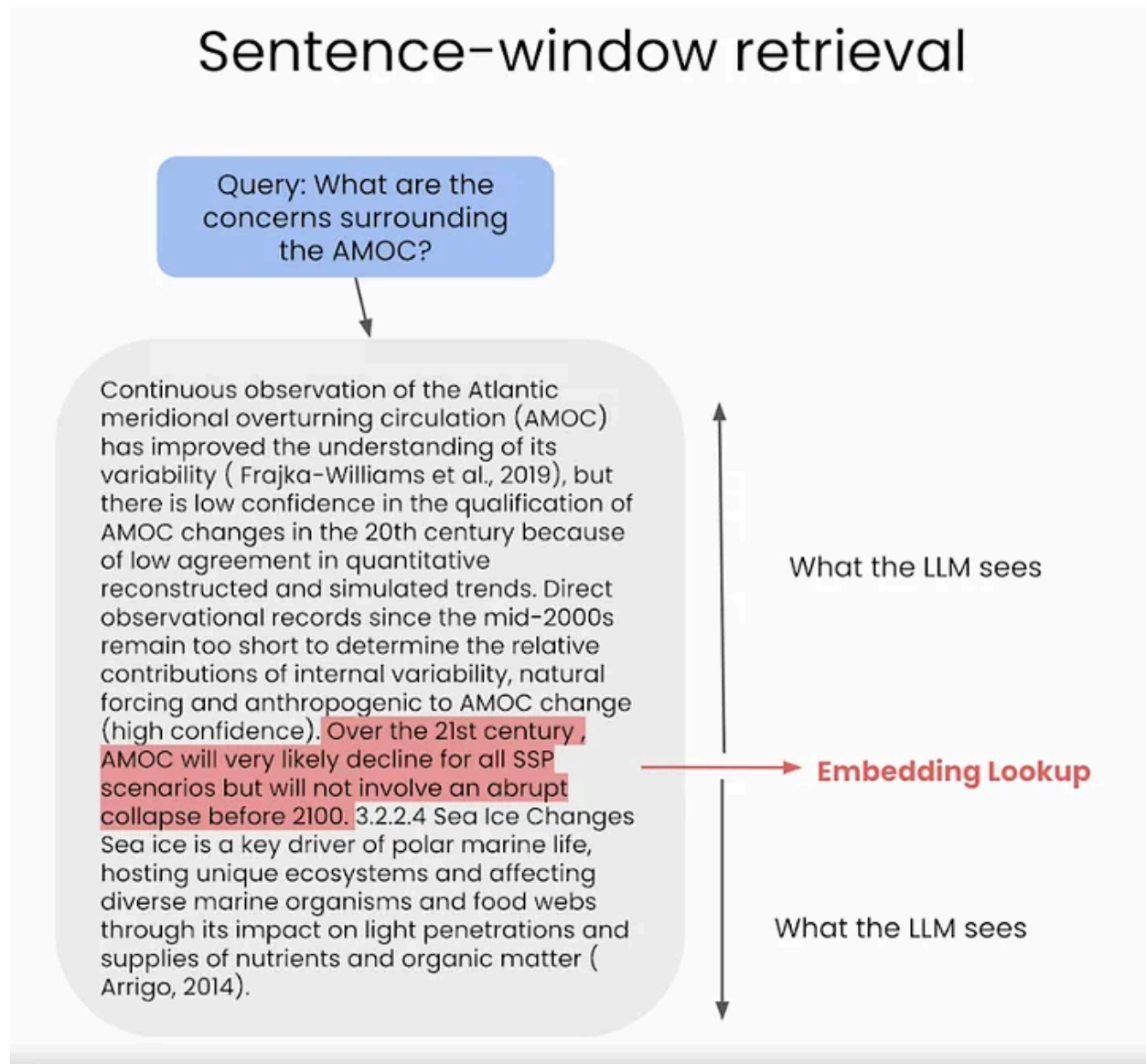
1. **Limited Contextual Understanding:** LLMs may require a larger window for synthesis to generate better responses, which this approach may not adequately provide.
2. **Potential for Suboptimal Responses:** Due to the limited context, the LLM might not have enough information to generate the most relevant and accurate responses.

2. Sentence-Window Retrieval / Small-to-Large Chunking

- The sentence-window approach breaks down documents into smaller units, such as sentences or small groups of sentences.
- It decouples the embeddings for retrieval tasks (which are smaller chunks stored in a Vector DB), but for synthesis it adds back in the context around the retrieved chunks.



During retrieval, we retrieve the sentences that are most relevant to the query via similarity search and replace the sentence with the full surrounding context (using a static sentence-window around the context, implemented by retrieving sentences surrounding the one being originally retrieved) as shown in the figure below



Advantages

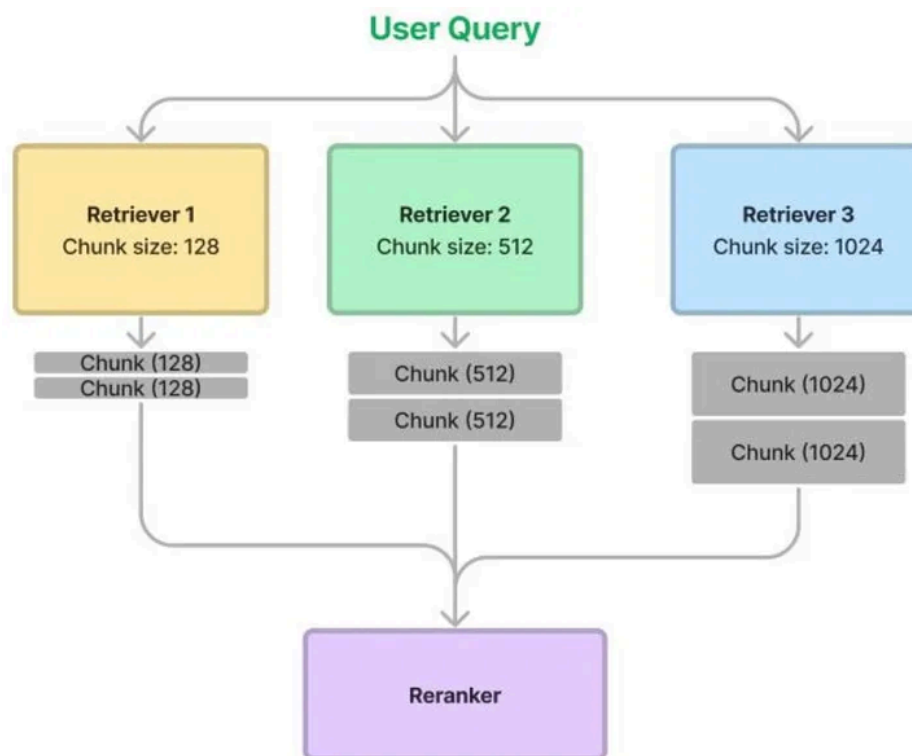
- **Enhanced Specificity in Retrieval:** By breaking documents into smaller units, it enables more precise retrieval of segments directly relevant to a query.
- **Context-Rich Synthesis:** It reintroduces context around the retrieved chunks for synthesis, providing the LLM with a broader understanding to formulate responses.
- **Balanced Approach:** This method strikes a balance between focused retrieval and contextual richness, potentially improving response quality.

Disadvantages

- **Increased Complexity:** Managing separate processes for retrieval and synthesis adds complexity to the pipeline.
- **Potential Contextual Gaps:** There's a risk of missing broader context if the surrounding information added back is not sufficiently comprehensive.

Retriever Ensembling and Reranking

- **Thought:** what if we could try a bunch of chunk sizes at once, and have a re-ranker prune the results?
- This achieves two purposes:
- Better (albeit more costly) retrieved results by pooling results from multiple chunk sizes, assuming the re-ranker has a reasonable level of performance.
- A way to benchmark different retrieval strategies against each other (w.r.t. the re-ranker).
- The process is as follows:
- Chunk up the same document in a bunch of different ways, say with chunk sizes: 128, 256, 512, and 1024.
- During retrieval, we fetch relevant chunks from each retriever, thus ensembling them together for retrieval.
- Use a re-ranker to rank/prune results.



- Based on evaluation results from LlamaIndex, faithfulness metrics go up slightly for the ensembled approach, indicating retrieved results are slightly more relevant. But pairwise comparisons lead to equal preference for both approaches, making it still questionable as to whether or not ensembling is better.
- Note that the ensembling strategy can be applied for other aspects of a RAG pipeline too, beyond chunk size, such as vector vs. keyword vs. hybrid search, etc.

Re-ranking

- Re-ranking in RAG refers to the process of evaluating and sorting the retrieved documents or information snippets based on their relevance to the given query or task.
- There are different types of re-ranking techniques used in RAG:
- Lexical Re-Ranking: This involves re-ranking based on lexical similarity between the query and the retrieved documents. Methods like BM25 or cosine similarity with TF-IDF vectors are common.
- Semantic Re-Ranking: This type of re-ranking uses semantic understanding to judge the relevance of documents. It often involves neural models like BERT or other transformer-based models to understand the context and meaning beyond mere word overlap.
- Learning-to-Rank (LTR) Methods: These involve training a model specifically for the task of ranking documents (point-wise, pair-wise, and list-wise) based on features extracted from both the query and the documents. This can include a mix of lexical, semantic, and other features.
- Hybrid Methods: These combine lexical and semantic approaches, possibly with other signals like user feedback or domain-specific features, to improve re-ranking.
- Neural LTR methods are most commonly used at this stage since the candidate set is limited to dozens of samples.

Some common neural models used for re-ranking are:

- Multi-Stage Document Ranking with BERT (monoBERT and duo BERT)
- Pretrained Transformers for Text Ranking BERT and Beyond
- ListT5
- ListBERT

Let's Understand Augmentation and Generation

Response Generation / Synthesis

- The last step of the RAG pipeline is to generate responses back to the user. In this step, the model synthesizes **(means combining or integrating different pieces of information from various sources.)** the retrieved information with its pre-trained knowledge to generate coherent and contextually relevant responses. This process involves integrating the insights gleaned from various sources, ensuring accuracy and relevance, and crafting a response that is not only informative but also aligns with the user's original query, maintaining a natural and conversational tone.

- Note that while creating the expanded prompt (with the retrieved top-k chunks) for an LLM to make an informed response generation, a strategic placement of vital information at the beginning or end of input sequences could enhance the RAG system's effectiveness and thus make the system more performant. This is summarized in the paper below.

Benefits of RAG:

- With RAG, the LLM is able to leverage knowledge and information that is not necessarily in its weights, providing it access to external knowledge bases.
- Improved relevance and accuracy.
- Handling open-domain queries.
- Reduced generation bias.
- Multi-model capabilities.
- Image Captioning, content summarization
- Human-AI Collaboration
- RAG doesn't require model retraining, saving time and computational resources.

In summary, RAG models are well-suited for applications where there's a lot of information available, but it's not neatly organized or labeled.

Disadvantages

RAG's performance depends on the comprehensiveness and correctness of the retriever knowledge base information loss.

If we look at the chain of processes in the RAG system.

1. Chunking the text and generating embedding for the chunks.
2. Retrieving the chunk by semantic similarity search.
3. Generate response based on the text of the top_k chunks.

Way of creating a RAG

- RAG libraries and frameworks
- Build your own RAG from scratch
- Langchain & LlamaIndex
- **Haystack:** End-to-end RAG framework for document search provided by Deepset.
- **REALM:** Retrieval Augmented Language Model (REALM) training is a Google toolkit for open-domain question answering with RAG.